



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



CC ZG506

Lecture Nos. 15 and 16

Agenda

Session 15

- IoT Analytics Life Cycle
- Pre-Processing in IoT
- Exploratory Data Analysis in IoT
- IoT Platforms - ThingsBoard
- AWS IoT

Session 16

- Course Review
- Q&A



IoT – Analytics Life Cycle

IoT-Analytics Life Cycle

THE IoT ANALYTICS LIFECYCLE



1. Generate

- Generate “telemetry” data from devices
- Ex: Temperature data, fuel level, air quality levels etc.

* “Telemetry data” refers to information collected from remote devices, systems, or sensors

2. Collect

- Data is collected, cached, and buffered at the edge for further processing or transmission
- Data collected from sensors or devices *are held in memory or storage buffers* until it can be processed or transmitted.

3. Analyze

- This step involves two actions:
 - a. Pre-process the data
 - b. Perform EDA (Exploratory Data Analysis)



Pre-Processing data in IOT

Data Quality Issues in IoT

1. Noise and Outliers
2. Missing data
3. Inconsistent data
4. Duplicate data
5. Data that is biased or unrepresentative of the phenomenon

Noise and Outliers

- Noise refers to random or irrelevant data points [Abnormal or Errors]
- Outliers are values that significantly deviate from the expected range [Valid but rare]

Ex: A temperature sensor in a factory occasionally reports a value of $1,000^{\circ}\text{C}$ due to an electrical fault, while the typical range is 20°C – 100°C . → Is this Noise or Outlier?

Techniques:

1. **Outlier Detection:** Use statistical methods (e.g., Z-score, IQR or Interquartile Range) or machine learning models to identify and filter out outliers.
2. **Redundancy:** Deploy multiple sensors to cross-check readings and identify anomalies.

Missing Data

- Data that is absent due to hardware malfunctions, network outages, or other reasons.
- Ex 1: A smart electricity meter fails to transmit data for several hours due to a network issue.
- Ex 2: A wearable health monitor stops collecting heart rate data due to a depleted battery.

Techniques:

1. **Imputation** : Fill missing values (mean, median or mode) or predictive models
2. **Alerts & Monitoring**: Set up alerts to quickly identify and resolve missing data issues.

Inconsistent Data

- Data that does not follow a consistent format, unit, or structure, often caused by device misconfiguration or differences between devices.
- Ex 1: One sensor records temperature in Celsius, while another records in Fahrenheit.
- Ex 2: Timestamp formats differ across devices

Techniques:

1. **Standardization:** Convert all the temperature into Celsius
2. **Transformation Pipeline:** Use ETL tools to convert to standard format ** [Next Slide]
3. **Validation Rules:** Implement validation checks during data ingestion to flag inconsistencies.

Inconsistent Data

ETL (Extract, Transform and Load)

- **Extract:** Devices send real-time events to a message broker or queue (e.g., Kafka, RabbitMQ).
- **Transform:** Stream processors (e.g., Flink, Kinesis) perform real-time transformations. This step helps in managing inconsistent data.
- **Load:** Transformed data is sent to the final destination (e.g., database, data warehouse, or dashboard).

We can use BI Tools like Tableau or Power BI to generate analytics or present dashboards.

Example for Standardizing Formats

Problem: IoT sensors send temperature data in different formats:

Sensor 1 sends data in JSON

```
{"sensor_id": 1, "temperature_celsius": 25.3}
```

Sensor 2 sends data in CSV

```
sensor_id,temperature_celsius  
2,27.8
```

Sensor 3 sends data in a nested JSON structure

```
{"id": 3, "readings": {"temp": 29.5}}
```

Goal

Standardize all incoming data into a consistent JSON format

```
{"sensor_id": 1, "temperature_celsius": 25.3}
```

```
{"sensor_id": 2, "temperature_celsius": 27.8}
```

```
{"sensor_id": 3, "temperature_celsius": 29.5}
```



Sample Apache Flink Code

[Processing data in real-time]

```
from pyflink.datastream import StreamExecutionEnvironment

def transform_record(record):
    # Handle JSON format
    if isinstance(record, dict) and "sensor_id" in record:
        return {"sensor_id": record["sensor_id"], "temperature_celsius": record["temperature_celsius"]}

    # Handle CSV format
    elif isinstance(record, str) and "," in record:
        sensor_id, temperature_celsius = record.split(",")
        return {"sensor_id": int(sensor_id), "temperature_celsius": float(temperature_celsius)}

    # Handle nested JSON format
    elif isinstance(record, dict) and "readings" in record:
        return {"sensor_id": record["id"], "temperature_celsius": record["readings"]["temp"]}

    # Default fallback
    return None

env = StreamExecutionEnvironment.get_execution_environment()

# Simulated stream of mixed-format data
stream = env.from_collection([
    {"sensor_id": 1, "temperature_celsius": 25.3},
    "2,27.8",
    {"id": 3, "readings": {"temp": 29.5}}
])

# Apply transformation
standardized_stream = stream.map(transform_record)

# Print standardized output
standardized_stream.print()

# Execute the pipeline
env.execute("Format Normalization Example")
```

Duplicate Data

- Repeated data entries caused by redundant transmissions or system errors.
- Ex 1: An environmental sensor sends the same reading multiple times due to a network glitch.
- Ex 2: A smart home system logs duplicate commands from a voice assistant (e.g., "Turn off the light").

Techniques:

1. **Deduplication Algorithms:** Use techniques like hashing or comparing unique identifiers (timestamps, device IDs) to remove duplicates.
2. **Edge Processing:** Perform deduplication at the edge before data is sent to the cloud.
3. **Rate-Limiting:** Implement rate-limiting at the device or gateway level to avoid duplicate transmissions.

Sometimes distinguishing duplicate data from valid data is tricky and challenging!

Biased or Unrepresentative Data

- Data that does not accurately represent the target population or phenomenon.
- Ex 1: A smart city project uses traffic data collected only from high-income neighborhoods, leading to biased urban planning decisions.
- Ex 2: Health data from wearables is skewed toward younger users, underrepresenting older populations.

Techniques:

1. **Sampling Strategy:** Ensure diverse and representative sampling of devices and locations. Stratified Sampling may be good approach.
2. **Diversity in Deployment:** Distribute IoT devices to cover varied environments and demographics.

Imputation Techniques

- Imputation techniques for IoT data are strategies used to handle missing, corrupt, or incomplete data collected from IoT devices.
- Since IoT devices often generate time-series data in real-time, missing data can arise due to device failures, network issues, or transmission errors.
- Proper imputation ensures the data's integrity and usability for analysis.

Imputation Techniques

Mean Imputation

- Replaces missing values with the **mean** of the observed values in the dataset.
- Use Case: Suitable for datasets where values fluctuate around a consistent range.
- Example: For temperature data, replace missing values with the mean temperature of the time-series data.

Imputation Techniques

Timestamp	Temperature (°C)
2024-11-21 12:00:00	22.5
2024-11-21 12:01:00	Missing
2024-11-21 12:02:00	23.0
2024-11-21 12:03:00	22.8
2024-11-21 12:04:00	Missing
2024-11-21 12:05:00	23.5

Mean = 22.95.

Replace missing values with the calculated Mean

Imputation Techniques

Median Imputation

- Replaces missing values with the **median** of observed values.
- Use Case: Ideal when the **data contains outliers**.
- Example: For air quality (e.g., PM2.5 levels), use the median of the sensor readings.

Imputation Techniques

- Dataset: 35.0, 42.0, **500.0** (outlier), 38.0, 40.0.
- **Mean**=131.0 $\mu\text{g}/\text{m}^3$. [Micrograms per Cubic Meter]
- **Median**: Middle value after sorting=40.0 $\mu\text{g}/\text{m}^3$
- The mean is heavily skewed by the outlier (500.0), while the median provides a more realistic imputed value.

Imputation Techniques

Mode Imputation

- Replaces missing values with the **most frequent value**.
- Use Case: Applicable for **categorical data**.
- Example: For “machine_status (ON/OFF)”, replace missing values with the most frequent state.

Imputation Techniques

Timestamp	Machine Status
2024-11-21 12:00:00	ON
2024-11-21 12:01:00	OFF
2024-11-21 12:02:00	Missing
2024-11-21 12:03:00	ON
2024-11-21 12:04:00	Missing
2024-11-21 12:05:00	ON
2024-11-21 12:06:00	ON

Observed Machine States: ON, OFF, Missing, ON, Missing, ON, ON

ON: 4 times occurrence

OFF: 1 time occurrence

MODE – ON [Most frequent state]

Characteristics of IoT Dataset

1. Dimensionality

- Number of attributes
- Curse of Dimensionality – the difficulties associated with analyzing high-dimensional data
- Dimensionality reduction techniques [Ex: PCA – Principal Component Analysis]

Example of high dimensionality

- Air pollutants [lets assume 50+ parameters]
- Primary dimensions-> PM2.5, PM10, NO₂, CO, O₃, SO₂, NH₃ etc.
- Meteorological data -> Temperature, Humidity, Wind Speed, Precipitation, etc.
- Geographic and Temporal data -> Latitude, Longitude, Altitude, time of day etc.

Characteristics of IoT Dataset

2. Sparsity

- For some data sets, such as those with asymmetric features, most attributes of an object have values of 0; in many cases, fewer than 1% of the entries are non-zero.
- Advantage because usually only the non-zero values need to be stored and manipulated.

Characteristics of IoT Dataset

Sparse Data												
Day	Sensor 1	Sensor 2	Sensor 3	Sensor 4	Sensor 5	Sensor 6	Sensor 7	Sensor 8	Sensor 9	Sensor 10	Sensor 11	Sensor 12
1-Jan	0	0.89	0	0	0	0	0	0	0	0	0	0.911
2-Jan	0	0	0	0	0	0	0	0	0	0	0	0.931
3-Jan	0	0.951	0	0	0	0	0	0	0	0	0	0.951
4-Jan	0.954	0.911	0	0	0	0	0	0	0	0	0	0.899
5-Jan	0	0	0	0	0	0	0	0	0	0	0	0.897
6-Jan	0	0.899	0	0	0	0	0	0	0	0	0	0.968
7-Jan	0.895	0.911	0	0	0	0	0	0	0	0	0	0.991
8-Jan	0.911	0.962	0	0	0	0	0	0	0	0	0	0.951
9-Jan	0	0.954	0	0	0	0	0	0	0	0	0	0.898
10-Jan	0.898	0.934	0	0	0	0	0	0	0	0	0	0.962

- All sensors are working
- Most of sensor outputs are zero
- High dimensionality, but less information
- Sparse data
- The values of Sensors 1, 2 and 12 need to be stored and processed

Characteristics of IoT Dataset

Missing Data												
Day	Sensor 1	Sensor 2	Sensor 3	Sensor 4	Sensor 5	Sensor 6	Sensor 7	Sensor 8	Sensor 9	Sensor 10	Sensor 11	Sensor 12
1-Jan	Missing	0.89	0	0	0	Missing	0	0	0	0	0	0.911
2-Jan	Missing	0	0	0	0	Missing	0	0	0	0	0	0.931
3-Jan	Missing	0.951	0	0	0	Missing	0	0	0	0	0	0.951
4-Jan	Missing	0.911	0	0	0	Missing	0	0	0	0	0	0.899
5-Jan	Missing	0	0	0	0	Missing	0	0	0	0	0	0.897
6-Jan	Missing	0.899	0	0	0	Missing	0	0	0	0	0	0.968
7-Jan	Missing	0.911	0	0	0	Missing	0	0	0	0	0	0.991
8-Jan	Missing	0.962	0	0	0	Missing	0	0	0	0	0	0.951
9-Jan	Missing	0.954	0	0	0	Missing	0	0	0	0	0	0.898
10-Jan	Missing	0.934	0	0	0	Missing	0	0	0	0	0	0.962

- Sensor 1 and Sensor 6 are malfunctioning
- Either fill the data manually, or discard the Sensor 1 and 6 data in calculations

Characteristics of IoT Dataset

3. Level of Resolution

- The patterns in the data depend on the level of resolution.
- If the resolution is too fine, a pattern may not be visible or may be buried in noise; if the resolution is too coarse, the pattern may disappear. [Ex: Air Pollution: Index (Chemical pollutants) measured per Second / Hour; per second – fine resolution; per hour – coarse resolution]



BITS Pilani
Pilani Campus



EDA in IOT

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) for IoT data involves uncovering patterns, relationships, and anomalies in the data generated by IoT devices.

Types:

1. Univariate Analysis
2. Multivariate Analysis
3. Temporal Analysis
4. Spatial Analysis

Exploratory Data Analysis (EDA)

Univariate Analysis

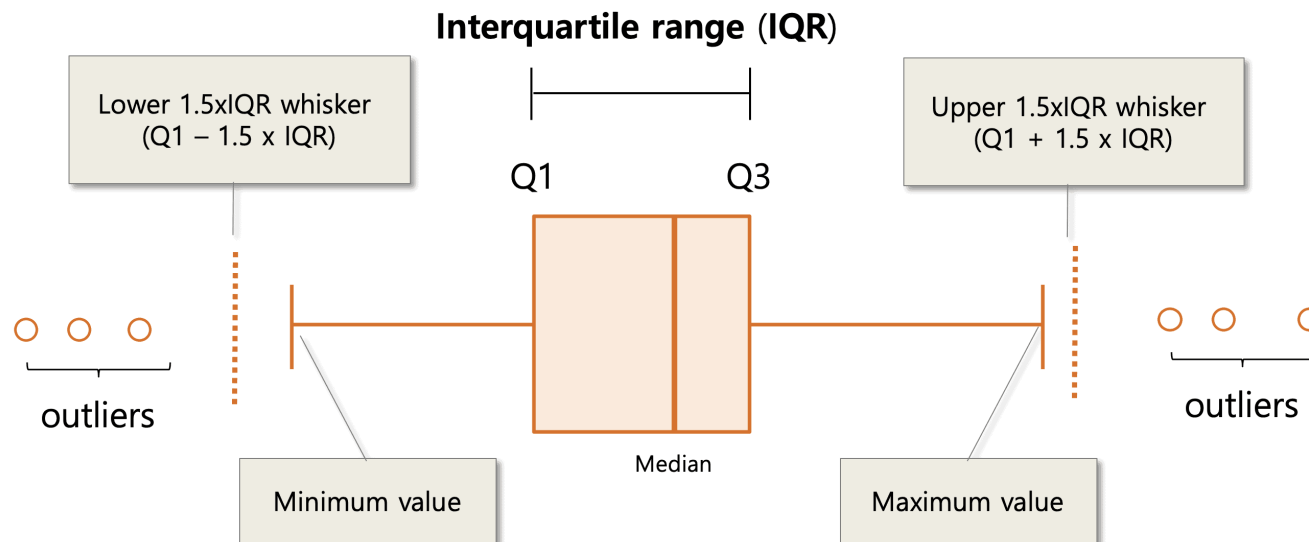
- Focuses on analyzing individual variables to understand their distribution and characteristics.
- Ex 1: Sensor Readings: Analyzing temperature, humidity, or pressure data from IoT sensors.
- Ex 2: Frequency of Events: How often an event (e.g., a device alert or failure) occurs.

Techniques:

1. Histograms to examine distributions.
2. Boxplots to identify outliers.
3. Summary statistics (mean, median, standard deviation).

EDA – BoxPlot [Whisker Plot]

- A boxplot incorporates the five-number summary.
- The ends of the box are at the quartiles.
- The box length is the interquartile range.
- The median is marked by a line within the box.
- The whiskers outside the box extend to the Minimum and Maximum observations.



IoT Example: Temperature Sensor Data

- Smart farming system with IoT sensors measuring temperature every hour.
- Collect this data for a day (24 readings) and measure the overall temperature pattern

Box Plot

- Minimum (Lowest Temp) - the coldest reading of the day.
- First Quartile (Q1) - 25% of the readings are below this. [less than or equal to Q1 in value]
- Median (Q2) - the middle value (half readings are below, half above).
- Third Quartile (Q3) - 75% of the readings are below this. [less than or equal to Q3 in value]
- Maximum (Highest Temp) - the hottest reading of the day.
- Outliers - any sudden spike/drop in temperature far from normal.

IoT Example: Temperature Sensor Data

Data Points -> [10, 12, 14, 16, 18, 20, 22, 24]

Five Point Summary

Minimum = 10

First Quartile (Q1) = average of 2nd and 3rd values = $(12 + 14) / 2 = 13$

Median (Q2) = average of 4th and 5th values = $(16 + 18) / 2 = 17$

Third Quartile (Q3) = average of 6th and 7th values = $(20 + 22) / 2 = 21$

Maximum = 24

EDA – BoxPlot [Whisker Plot]

Suppose we have a dataset of air quality PM2.5 readings:

- Median: 40
- Q1: 30
- Q3: 50
- $IQR = 50 - 30 = 20$

The whiskers would extend to:

- Lower bound - $Q1 - 1.5 * IQR = 30 - 1.5 * 20 = 0$
- Upper bound - $Q3 + 1.5 * IQR = 50 + 1.5 * 20 = 80$
- Any values below 0 or above 80 would be classified as outliers and plotted as individual points.

EDA – BoxPlot [Whisker Plot]

Consider the list of observations for 'Temperature' sensor.

22.5, 22.8, 23.1, 23.5, 24.0, 24.2, 24.6, 25.0, 25.2, 25.5, 25.8, 26.0, 26.5, 26.8,
27.0, 27.2, 27.5, 27.8, 28.0, 45.5

Draw a box-plot to represent the above data.

Median, Minimum, Maximum, First quartile, Third quartile, Interquartile Range

BoxPlot Calculator - <http://www.alcula.com/calculators/statistics/box-plot/>

Exploratory Data Analysis (EDA)

Multivariate Analysis

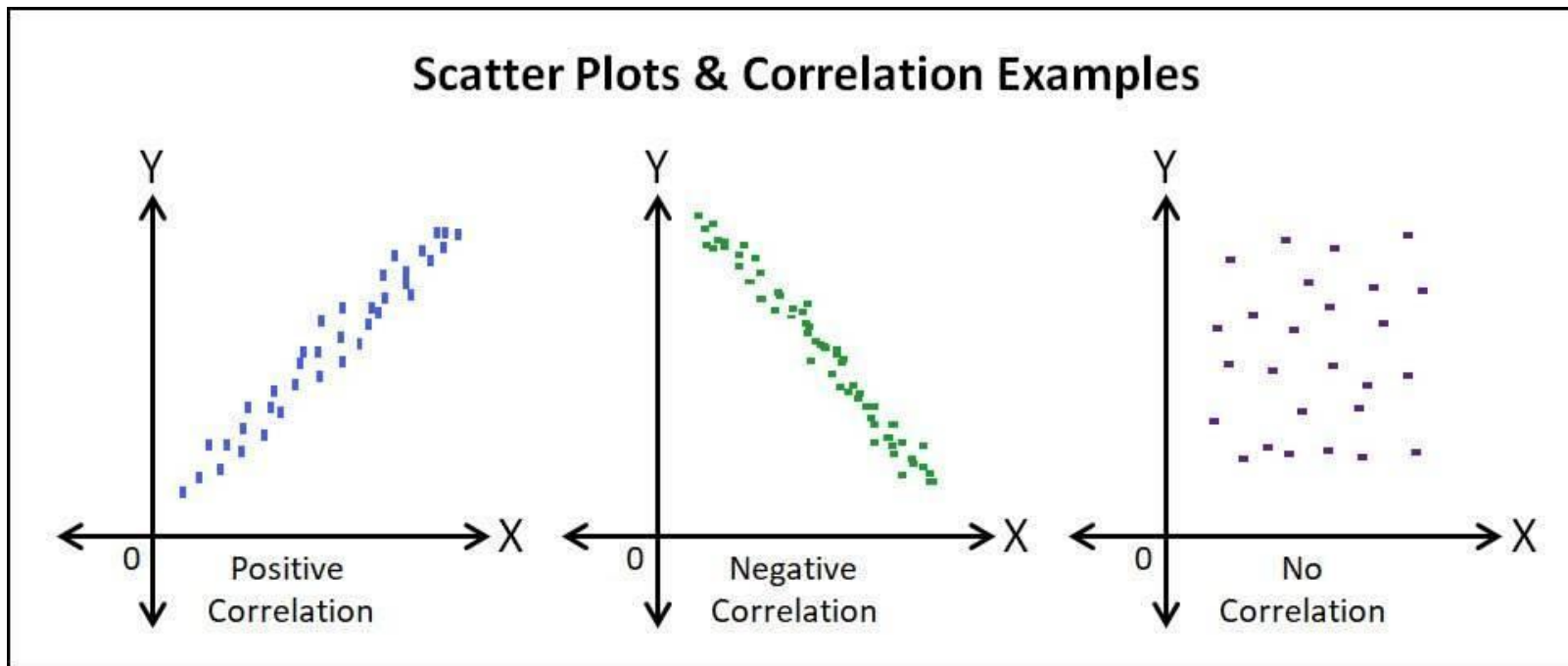
- Examines relationships between multiple variables.
- Ex 1: Correlation between temperature and humidity in a smart greenhouse.
- Ex 2: Relationship between vehicle speed and engine vibration in connected cars.

Techniques

1. Scatter plots for visualizing variable relationships
2. Heatmaps for correlation matrices

EDA – ScatterPlot

- Determine if there appears to be a relationship, pattern, or trend between two numeric attributes.
- We can use “Pearson Correlation” for this



Exploratory Data Analysis (EDA)

Temporal Analysis

- IoT data often has a **strong time dimension** due to continuous data streams.
- Ex 1: Daily energy usage patterns from smart meters.
- Ex 2: Seasonal variations in traffic data from smart city sensors.

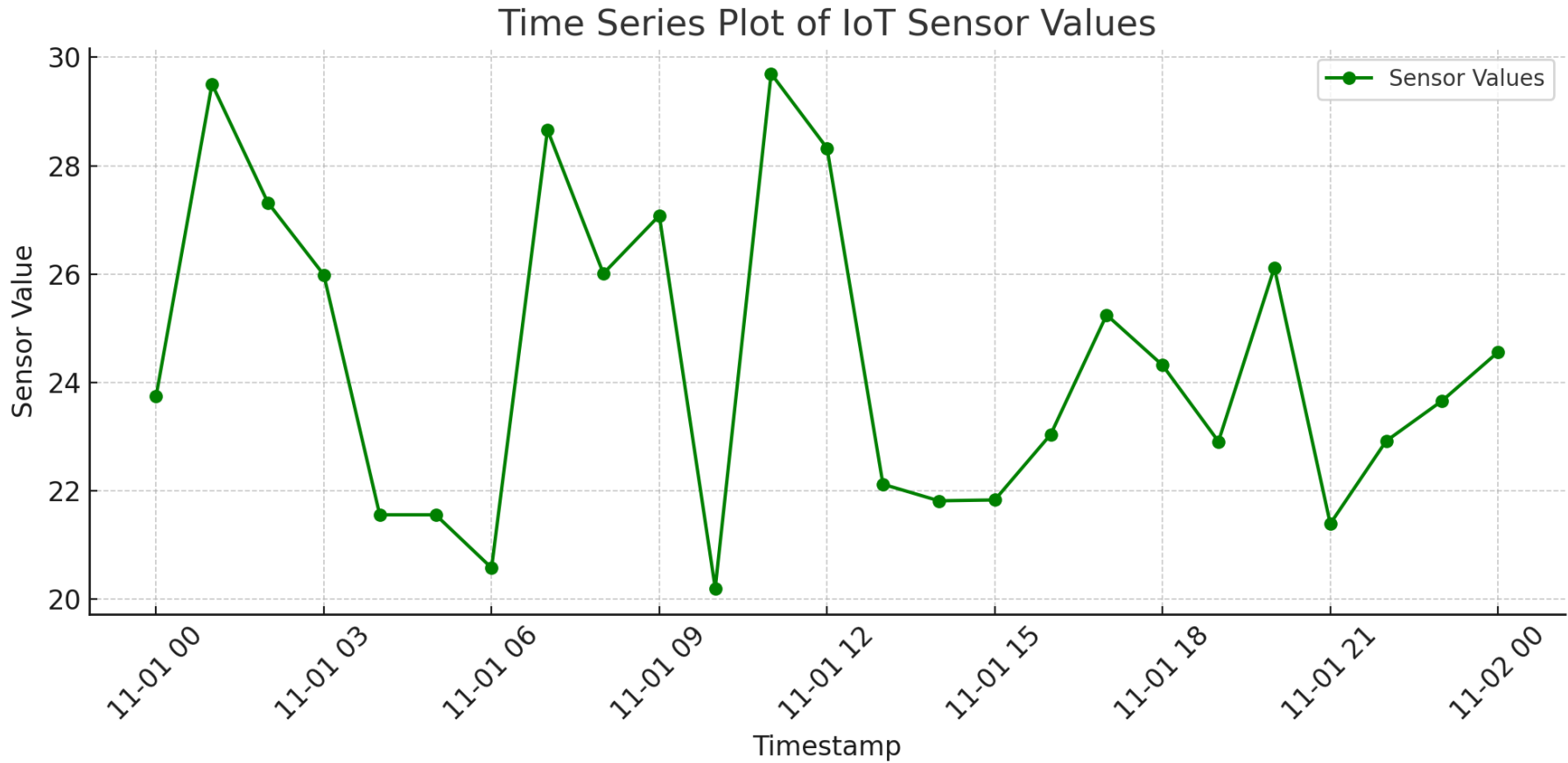
Techniques

1. Time series plots to observe trends and seasonality.
2. Decomposition to separate trends, seasonality, and noise

Dataset

Timestamp	SensorValue
11/1/2024 0:00	23.74540119
11/1/2024 1:00	29.50714306
11/1/2024 2:00	27.31993942
11/1/2024 3:00	25.98658484
11/1/2024 4:00	21.5601864
11/1/2024 5:00	21.5599452
11/1/2024 6:00	20.58083612
11/1/2024 7:00	28.66176146
11/1/2024 8:00	26.01115012
11/1/2024 9:00	27.08072578
11/1/2024 10:00	20.20584494
11/1/2024 11:00	29.69909852
11/1/2024 12:00	28.32442641
11/1/2024 13:00	22.12339111
11/1/2024 14:00	21.81824967
11/1/2024 15:00	21.8340451
11/1/2024 16:00	23.04242243
11/1/2024 17:00	25.24756432
11/1/2024 18:00	24.31945019
11/1/2024 19:00	22.9122914
11/1/2024 20:00	26.11852895
11/1/2024 21:00	21.39493861
11/1/2024 22:00	22.92144649
11/1/2024 23:00	23.66361843
11/2/2024 0:00	24.56069984

Time Series Data



Exploratory Data Analysis (EDA)

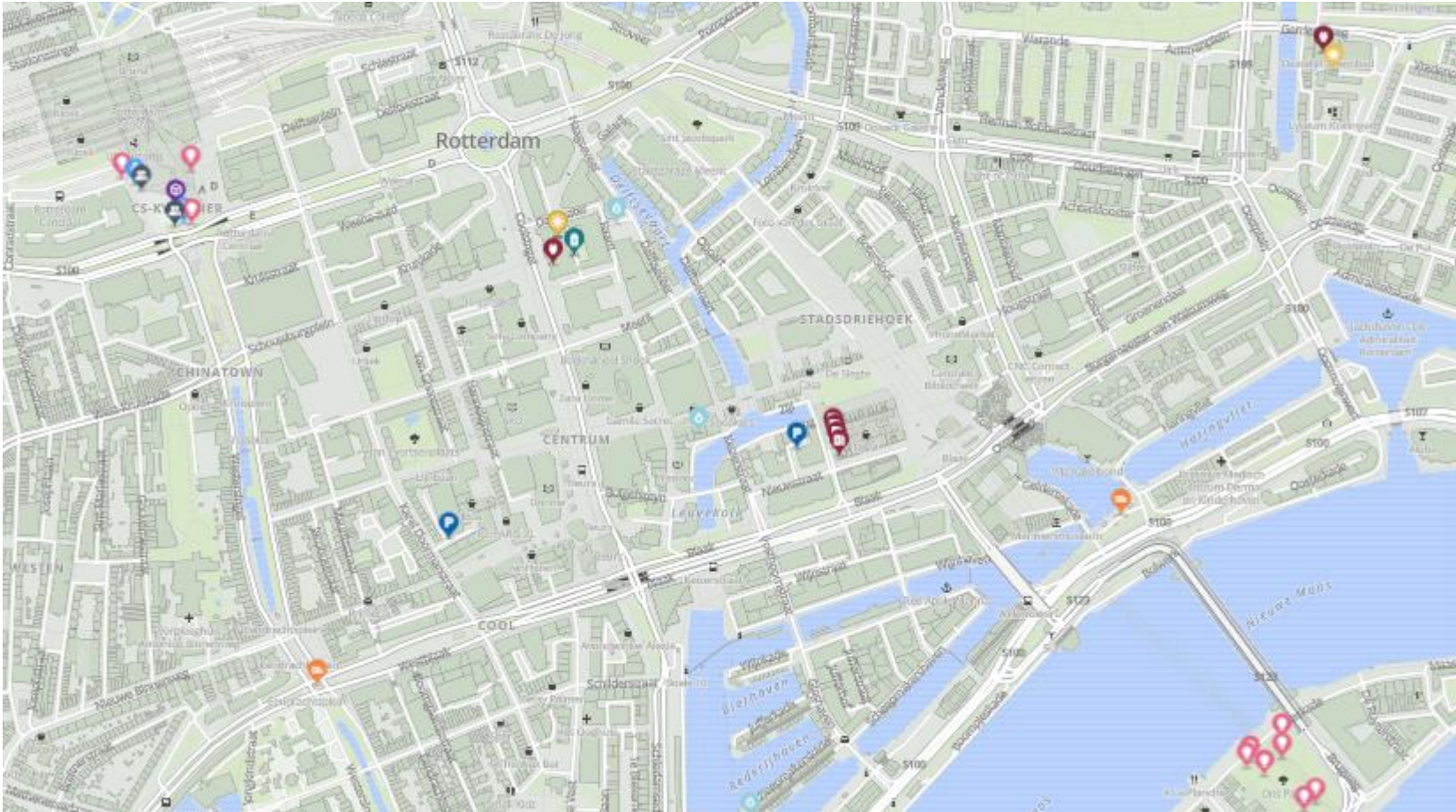
Spatial Analysis

- IoT data often comes with geolocation information, enabling spatial analysis.
- Ex 1: Mapping air quality measurements from sensors across a city.
- Ex 2: Analyzing GPS data from delivery trucks to optimize routes.

Techniques

1. Geospatial heat maps to visualize sensor data distributions.
2. Cluster analysis to identify hotspots or anomalies.

OpenRemote – GeoSpatial Map



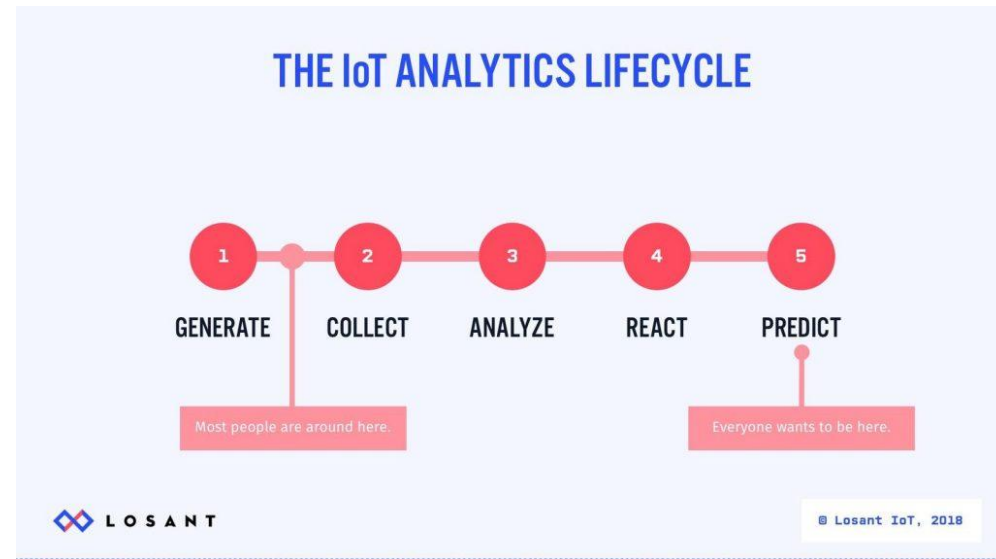
Exploratory Data Analysis (EDA)

Tools for EDA in IoT

- **Python:** Pandas, Matplotlib, Seaborn, Plotly, Scikit-learn.
- **Visualization Platforms:** Tableau, Power BI, Grafana.
- **IoT-Specific Platforms:** AWS IoT Analytics, Azure IoT, Google Cloud IoT.

4. React

- Automatically making real-time decisions from your data that feed back into a business process
- Ex 1: When faults are detected by your equipment, your environment can react by automatically generating a support ticket in another system.
- Ex 2: In office environments, conference rooms can be automatically freed up if no motion is detected in a booked room.



5. Predict

- Perform Predictive Analytics
- Ex: Predict the air quality for the next day, by considering the previous two days, for a region.



BITS Pilani
Pilani Campus



ThingsBoard IoT Platform



ThingsBoard APIs

1. Device APIs

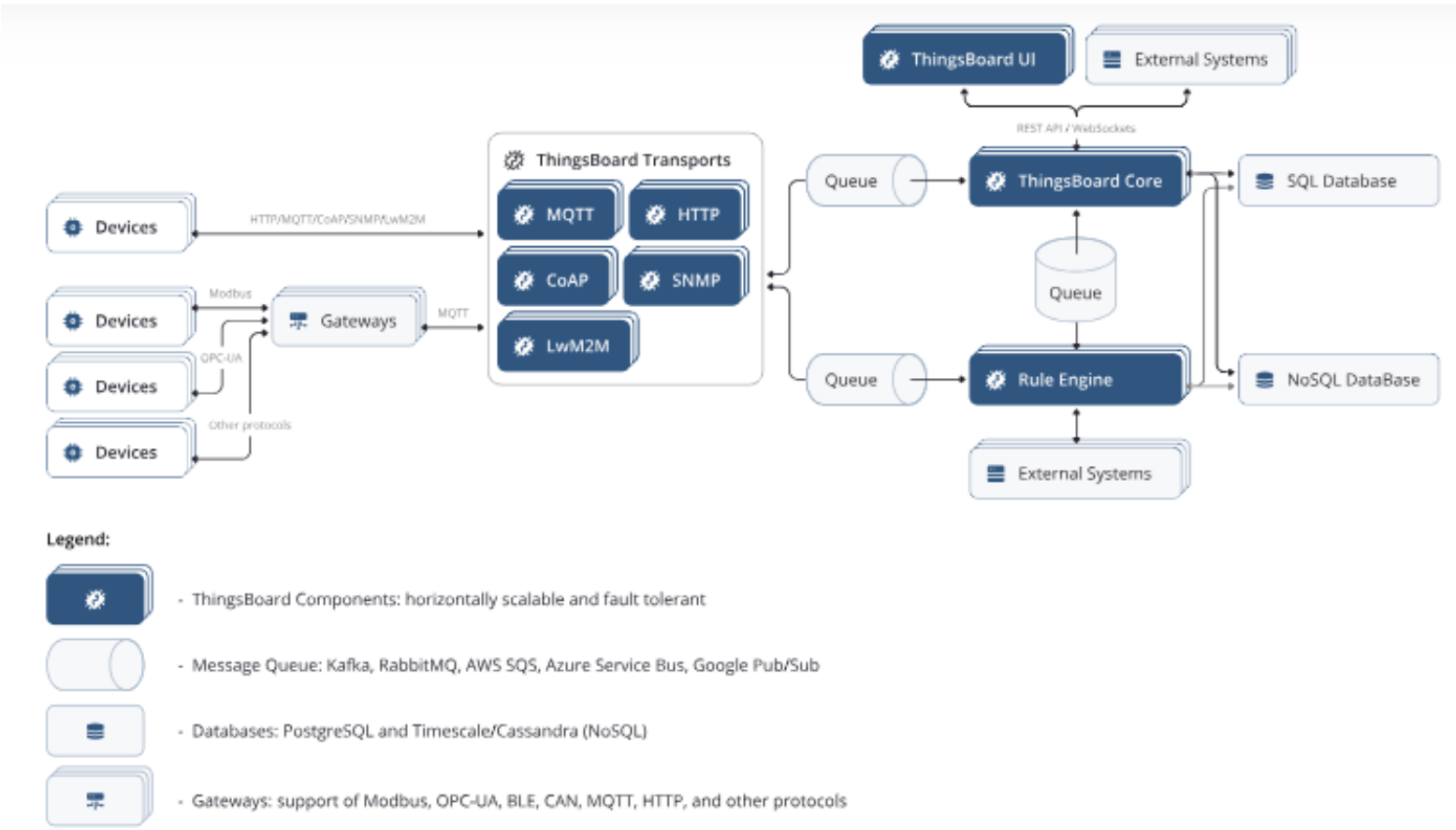
- MQTT API
- CoAP API
- HTTP API

2. Server-Side APIs

- Administration REST API
- RPC API
- REST Client

<https://thingsboard.io/docs/api/>

ThingsBoard Architecture



<https://thingsboard.io/docs/reference/>

ThingsBoard

Mini Demo [To get started]

1. Build a “Air Quality Monitoring” system with 1 device.
2. Use POST API for sending data (HTTP) to the device

URL:

1. Docker Installation - <https://thingsboard.io/docs/user-guide/install/docker-windows/>

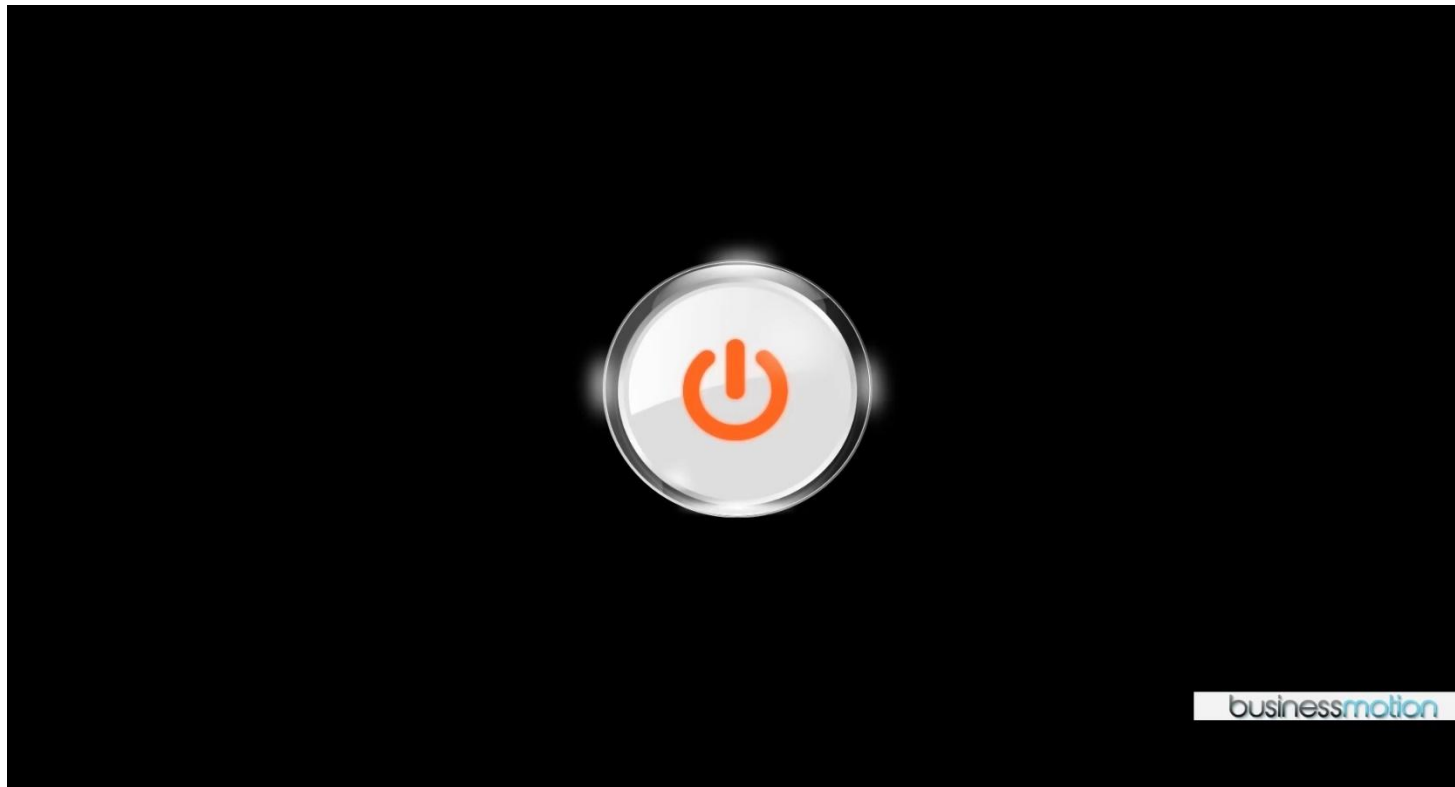


BITS Pilani
Pilani Campus

AWS IoT Platform

AWS IoT Architecture

Real-Time IoT Device Monitoring with Kinesis Data Analytics



<https://www.youtube.com/watch?v=uWUAcc68MWI>

Available as IaC CloudFormation template



AWS IoT Components



AWS IoT Components

Benefits and features

Kinesis Data Streams

Real-time data capture

Ingest and store data streams from hundreds of thousands of data sources:

- Log and event data collection
- IoT device data capture
- Mobile data collection
- Gaming data feed

Amazon Data Firehose

Load real-time data

Load streaming data into data lakes, data stores, and analytics tools for:

- Log and event analytics
- IoT data analytics
- Clickstream analytics
- Security monitoring

Managed Apache Flink

Get insights in real time

Analyze streaming data and gain actionable insights in real time:

- Real-time streaming ETL
- Real-time log analytics
- Ad tech and digital marketing analytics
- Real-time IoT device monitoring



Course Review & Discussion

Module Structure



Module	Description
0	Python Programming [Self-Study]
1	API Basics
2	Cloud Native Application
3	Data Science and Machine Learning
4	AI and Cognitive Services Part A: Basic APIs for AI applications Part B: Language Models Part C: Cloud Native Application Deployment
5	IoT and Data Analytics



Contact Session wise Coverage

Session	Coverage	
CS01	API Basics (OpenAPI, REST, gRPC, GraphQL)	} Basics
CS02	API Basics + Cloud Native Application	
CS03	Cloud Native Application	
CS04	Introduction to Data Science & Machine Learning, DataOps & MLOps	} Data Science & ML
CS05	Introduction to Data Science & Machine Learning, DataOps & MLOps	
CS06	API driven Cloud-native Data Pipeline	
CS07	API-driven ML pipelines	} Cognitive AI
CS08	Basic APIs for AI Applications - NLP, CV, Speech recognition	
CS09	Language Models (LLMs)	
CS10	Language Models (LLMs / SLMs)	
CS11	Open AI APIs + Fine Tuning	
CS12	RAG Models	} IoT
CS13	Cloud Native Application Deployment	
CS14	IoT and Data Analytics	
CS15	IoT and Data Analytics	
CS16	Course review and discussion	

Experiential Learning

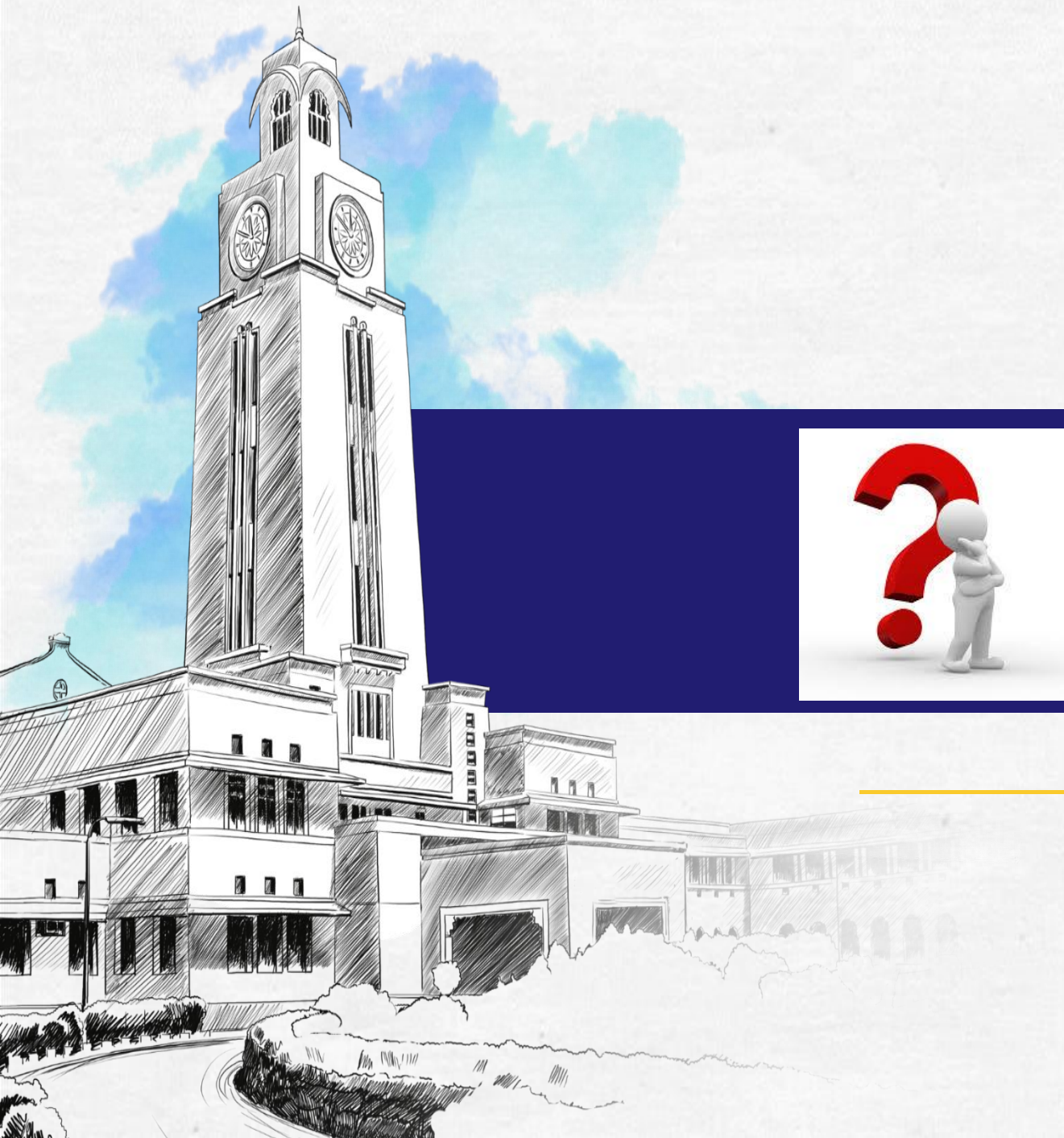


Lab. No	Module No.	Session No.	Experiment	Tools list
1	1	1,2	Demonstrate REST, GraphQL and gRPC APIs	REST -> FastAPI GraphQL -> Strawberry library in Python
2	3	5	Demonstrate the design and development of API-based data pipeline which involves data ingestion, pre-processing, analysis and monitoring activities	Prefect Cloud
3	3	7	Demonstrate the design and development of Machine Learning pipeline which involves model development, training, deployment and monitoring activities	MLFlow AWS SageMaker
4	4	8 to 10	Demonstrate NLP tasks such as speech processing, language translation, text summarization, and text generation using Cloud-based APIs; Demonstrate Fine Tuning	HuggingFace APIs for LLM and SLM Open AI APIs
5	4	11 and 12	Demonstrate a generative AI assistant, using RAG approach, which answers queries about an application (chatbot)	LangChain Python code to invoke APIs Open AI APIs
6	5	14 and 15	Demonstrate an IoT and Data Analytics application	OpenRemote ThingsBoard

Evaluation



Evaluation Component	Name	Type	Weightage	Comments
EC - 1	Quiz I	Online	5%	Module 1 & 2
	Assignment I	Online	15%	Module 3
	Assignment II	Online	15%	Module 4
EC - 2	Mid-term Exam	Closed book	30%	Modules 1,2,3 2 hours
EC - 3	End Semester Exam	Open book	35%	Modules 1,2,3,4,5 2.5 hours



BITS Pilani
Pilani | Dubai | Goa | Hyderabad





BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Revision (Self Study)

Preparation for Comprehensive Exam



Thank You!